Monitor de Riesgo Académico Saber 11: Modelos y Tablero

Proyecto — Entrega 2 · Despliegue de Soluciones Analíticas · 8 de noviembre de 2026

Mateo Ríos
Maestría en Inteligencia Analítica de Datos
Universidad de los Andes
Bogotá, Colombia

Harol Perafan
Maestría en Inteligencia Analítica de Datos
Universidad de los Andes
Bogotá, Colombia

Sebastian Vivas
Maestría en Inteligencia Analítica de Datos
Universidad de los Andes
Bogotá, Colombia

***Resumen*—Se entrenaron y compararon 10 modelos de clasificación del riesgo de bajo desempeño en Saber 11 y 3 modelos de regresión del puntaje global, registrados en un servidor MLflow desplegado en AWS EC2. El mejor modelo, un gradient boosting basado en histogramas, alcanza AUC-ROC de 0,800 y AUC-PR de 0,589 en el conjunto de prueba; con un umbral elegido para detectar al menos 70 % de los estudiantes en riesgo, su precisión es 0,48, 1,9 veces la tasa base. El modelo se integró en un tablero Dash con diagnóstico territorial, predicción individual explicada y priorización por lotes.**

***Palabras clave*—Saber 11, riesgo académico, gradient boosting, MLflow, AWS EC2, tablero Dash.**

I. Resumen del problema

Contexto. El puntaje global de Saber 11 (0–500) condiciona el acceso a la educación superior y a becas, y presenta brechas marcadas por territorio y contexto socioeconómico (p. ej., 61 % de estudiantes en el cuartil inferior en Chocó frente a 12 % en Bogotá). Las secretarías de educación focalizan hoy sus programas de refuerzo con resultados del año anterior, cuando el estudiante ya presentó la prueba.

Pregunta de negocio y alcance. ¿Qué estudiantes (y colegios/municipios) tienen mayor riesgo de obtener un puntaje global bajo (< 210, percentil 25 nacional), dadas sus características socioeconómicas y del colegio, para priorizar intervenciones de refuerzo? El alcance es un prototipo con modelos supervisados, una API de inferencia y un tablero, desplegados con Docker en AWS.

Datos. Resultados únicos Saber 11 del ICFES (datos.gov.co, id kgxf-xxbe) [1], periodo 20224: 532.788 estudiantes únicos, 10.378 colegios, versionados con DVC en S3.

Cambios respecto a la Entrega 1.

- *Integración modelo–tablero:* en esta entrega el tablero carga el modelo empaquetado exportado desde MLflow; la API FastAPI y los contenedores Docker se abordan en la Entrega 3.
- *Variables:* se fijó un conjunto de 19 variables previas a la prueba; la edad, no destacada en E1, resultó la segunda más importante (captura la extraedad).
- *Umbral de decisión:* se calibró en 0,26 sobre validación para cumplir el criterio de recall ≥ 0,70 definido en E1.
- *Tablero:* la pestaña «Priorización de colegios» pasó a «Priorización por lotes» (CSV) y el ranking de colegios se integró al diagnóstico al filtrar un municipio.

II. Desarrollo de los modelos

A. Variables y selección de características

Se usan solo variables conocidas antes de la prueba (src/features.py): nueve del hogar (educación de madre y padre, estrato, internet, computador, automóvil, lavadora, personas y cuartos), ocho del colegio (naturaleza, zona, jornada, calendario, bilingüismo, carácter, género y departamento), el género y la edad del estudiante. Los puntajes por área se excluyen porque determinan el puntaje global (fuga de información). La selección se apoyó en el η^2 del EDA y se validó con una ablación que entrena el mejor algoritmo solo con variables del hogar o solo del colegio.

B. Preprocesamiento y protocolo

Los nulos categóricos (4–18 %) se codifican como la categoría «Sin información» y la edad se imputa con la mediana. Los modelos lineales usan one-hot y los de árboles codificación ordinal; el gradient boosting usa además el soporte nativo de variables categóricas. Los datos se dividen de forma estratificada en entrenamiento (70 %), validación (15 %) y prueba (15 %) con semilla fija. La selección de modelos usa solo validación: el criterio principal es el AUC-PR, más informativo que el AUC-ROC con clases desbalanceadas [7], y el umbral se elige como el mayor valor con recall ≥ 0,70 en validación.

C. Algoritmos y experimentos

Se evaluaron regresión logística (con y sin ponderación de clases y dos niveles de regularización), bosque aleatorio [5] y gradient boosting basado en histogramas de scikit-learn [3], inspirado en LightGBM [4], con varias configuraciones de tasa de aprendizaje, iteraciones y hojas. Para el puntaje global se comparó Ridge con gradient boosting de regresión. La Tabla I resume los experimentos.

Tabla I
Experimentos registrados en MLflow

Run	Algoritmo y parámetros
logreg-base	Reg. logística, C = 1
logreg-C0.1-bal / C1-bal	Reg. logística, class_weight = balanced, C ∈ {0,1; 1}
rf-d14	Bosque, 100 árboles, prof. 14, 30 % de muestra
rf-d20-bal	Bosque, prof. 20, hoja ≥ 10, balanceado
hgb-lr0.1	Gradient boosting, lr = 0,1, 200 iteraciones
hgb-lr0.05-l63(-bal)	lr = 0,05, 500 iter., 63 hojas (± balanceo)

hgb-solo-hogar / colegio	Ablación de variables con hgb-lr0.05-l63
ridge-a1 / hgbr-*	Regresión del puntaje global

D. Seguimiento con MLflow

Cada ejecución de src/train.py crea un run en un servidor MLflow [2] en una instancia EC2 (http://54.242.29.105:5000) con backend SQLite y almacenamiento de artefactos. Por run se registran parámetros, métricas de validación y prueba, curvas ROC/PR, matriz de confusión, importancia por permutación y el modelo con su firma. src/select_best.py registra el mejor run de cada experimento en el Model Registry (alias «champion») y lo exporta a models/, versionado con DVC.

III. Evaluación de los Modelos

Tabla II
Clasificación: Métricas en Validación

Run	AUC-ROC	AUC-PR	Precisión*
hgb-lr0.05-l63	0,800	0,589	0,481
hgb-lr0.05-l63-bal	0,800	0,589	0,481
hgb-lr0.1	0,800	0,589	0,479
rf-d20-bal	0,780	0,550	0,448
logreg-base	0,778	0,546	0,448
rf-d14	0,775	0,544	0,441
logreg-C1-bal	0,780	0,544	0,450
logreg-C0.1-bal	0,780	0,544	0,450
hgb-solo-hogar	0,761	0,531	0,422
hgb-solo-colegio	0,754	0,513	0,412

** Precisión con el umbral que garantiza recall ≥ 0,70.*

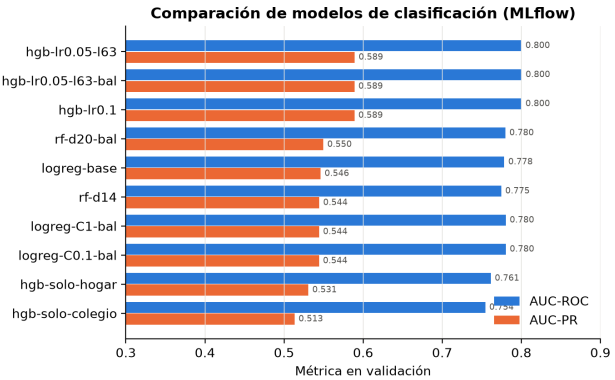


Fig. 1. Comparación de los modelos de clasificación en validación (generada desde MLflow).

El gradient boosting domina (Tabla II, Fig. 1): hgb-lr0.05-l63 obtiene AUC-PR de 0,589 frente a 0,550 del mejor bosque y 0,546 de la regresión logística. Se seleccionó como campeón y se evaluó en prueba (Tabla III).

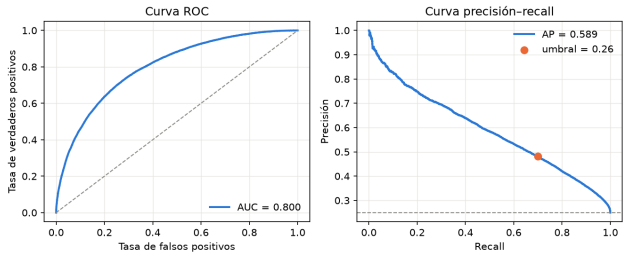


Fig. 2. Curvas ROC y precisión–recall del modelo campeón (hgb-lr0.05-l63) en validación; el punto naranja marca el umbral de decisión.

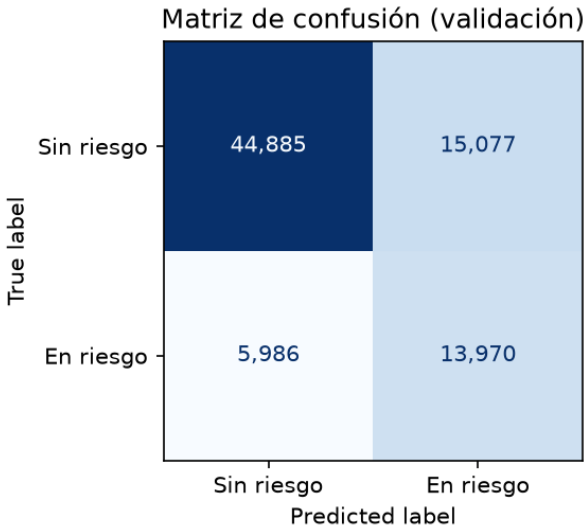


Fig. 3. Matriz de confusión del modelo campeón con el umbral de decisión (validación).

Tabla III
Modelos Campeones en el Conjunto de Prueba

Modelo	Métrica	Valor
hgb-lr0.05-l63	AUC-ROC / AUC-PR	0,800 / 0,589
	Recall / Precisión / F1	0,70 / 0,48 / 0,57
	Umbral	0,26
hgbr-lr0.05-l63	RMSE / MAE (puntos)	40,7 / 32,8
	R^2	0,38

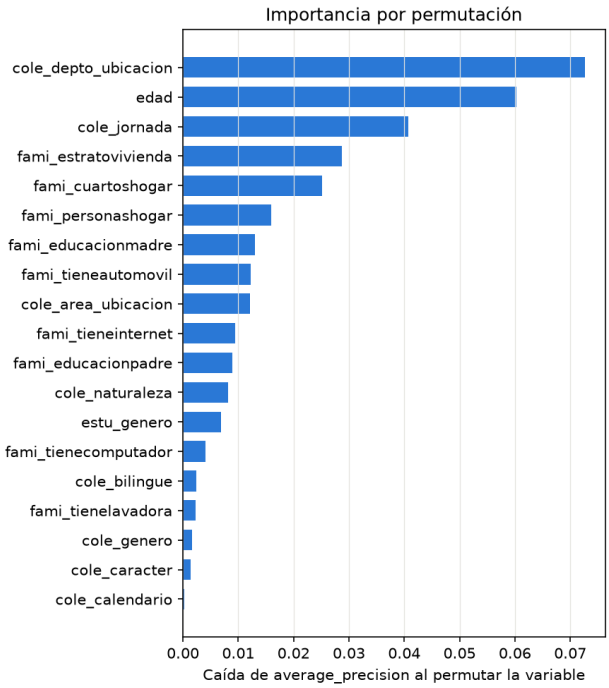


Fig. 4. Importancia por permutación (caída del AUC-PR) del modelo campeón.

TABLA IV
REGRESIÓN DEL PUNTAJE GLOBAL (VALIDACIÓN)

Run	RMSE	MAE	R^2
hgbr-lr0.05-l63	40,8	32,9	0,379
hgbr-lr0.1	40,8	33,0	0,378
ridge-a1	42,8	34,8	0,315

IV. OBSERVACIONES Y CONCLUSIONES

- *Algoritmo:* el boosting supera a los modelos lineales y al bosque en ≈ 0,02 de AUC-ROC y ≈ 0,04 de AUC-PR, porque captura interacciones (p. ej., departamento × zona × jornada). Entre sus configuraciones la diferencia de AUC-PR es de apenas 0,0003: el desempeño está limitado por la información de las variables, no por la capacidad del modelo.
- *Desbalance:* ponderar las clases no cambia el AUC (la capacidad de ordenar estudiantes) y solo desplaza el umbral; ajustar el umbral en validación es suficiente y más transparente.
- *Selección de variables:* con solo variables del hogar el AUC-ROC cae a 0,761 y con solo variables del colegio a 0,754, frente a 0,800 con todas: ambos grupos son complementarios (Fig. 1).
- *Factores:* departamento, edad y jornada dominan la importancia (Fig. 4). La edad captura la extraedad, coherente con el alto riesgo de las jornadas nocturna y sabatina del EDA. La educación de la madre pierde peso individual porque comparte información con estrato, cuartos y recursos del hogar.
- *Generalización:* las métricas de prueba (AUC-ROC 0,800) son prácticamente iguales a las de validación (0,800), sin señales de sobreajuste.
- *Valor operativo:* con recall 0,70, de cada 100 estudiantes priorizados ≈ 48 están realmente en riesgo, frente a 25 si se eligieran al azar (1,9× más eficiente) (Figs. 2–3). El criterio de éxito de E1 (AUC ≥ 0,75 y recall ≥ 0,70) se cumple.
- *Puntaje estimado:* el regresor explica 38 % de la varianza (RMSE 40,7); es un complemento orientativo y la decisión se basa en la probabilidad de riesgo.
- *Limitaciones:* un solo periodo (20224), estrato autorreportado y variables sensibles (departamento, género): el modelo debe usarse para priorizar apoyo, no para etiquetar estudiantes. Se validará la estabilidad temporal con el periodo 20194 en la siguiente entrega.

V. TABLERO DESARROLLADO

El tablero (dashboard/app.py, Dash [6]) sigue la maqueta de la Entrega 1 y responde la pregunta de negocio en tres pestañas. (1) Diagnóstico territorial: filtros por departamento, municipio, naturaleza, zona y jornada; KPIs; tasa de riesgo por departamento (un clic filtra y baja a municipios); factores con selector de variable; tabla de municipios o colegios prioritarios; y la matriz estrato × naturaleza. (2) Predicción de riesgo: un formulario con el perfil del estudiante devuelve la probabilidad, el nivel (bajo/medio/alto), el puntaje estimado, la tasa de su departamento, una recomendación y los factores que más pesan, calculados por oclusión frente al perfil más común; incluye las métricas del modelo registrado. (3) Priorización por lotes: se carga un CSV de estudiantes, se obtiene la lista ordenada por riesgo y se descarga el resultado.

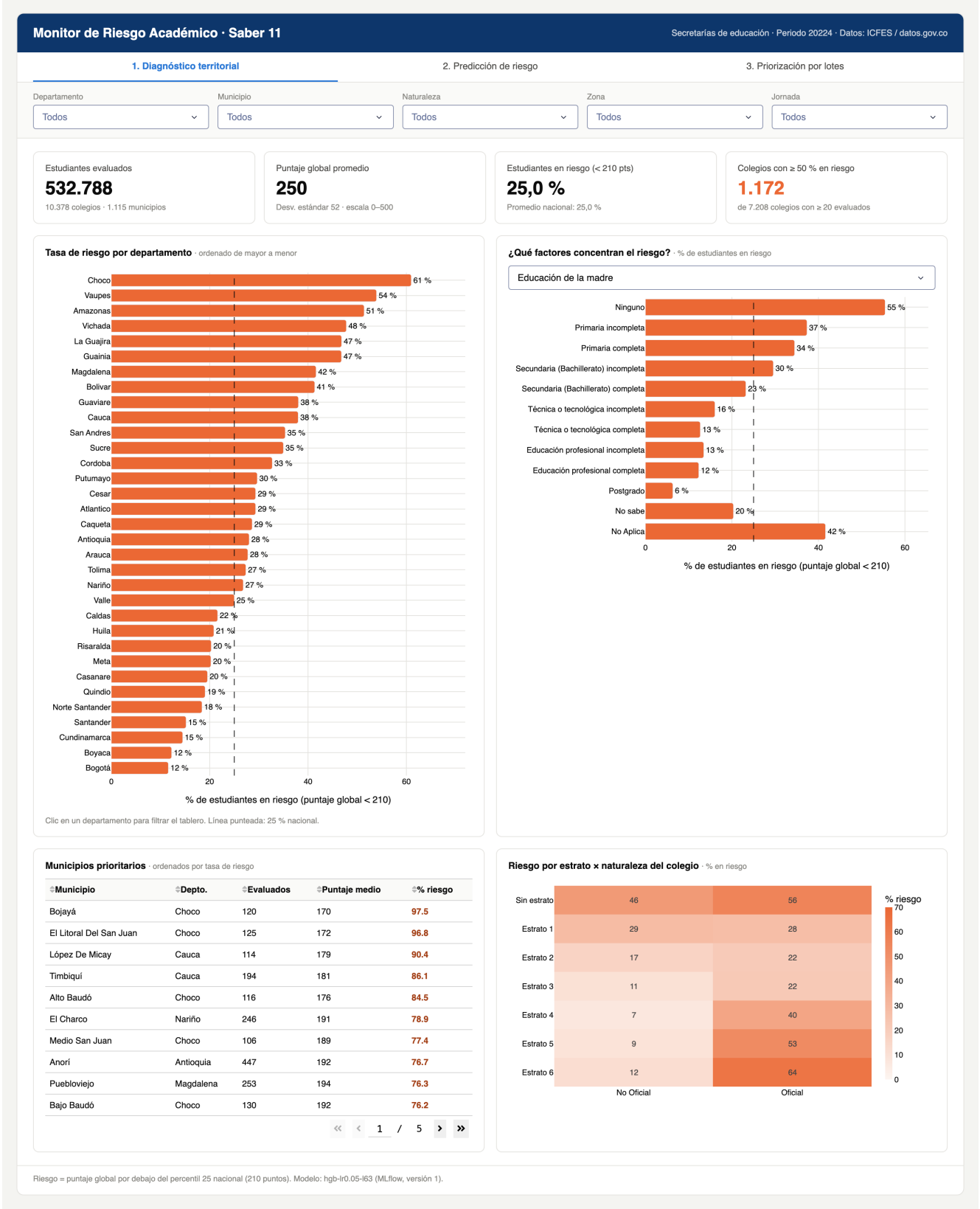


Fig. 5. Pestaña 1 — Diagnóstico territorial.

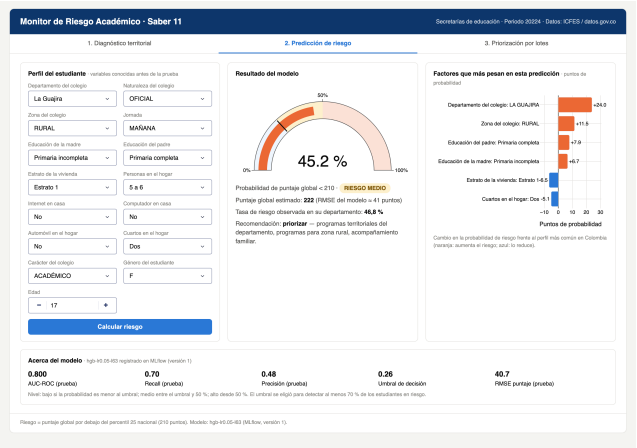


Fig. 6. Pestaña 2 — Predicción de riesgo con factores explicativos y métricas del modelo.

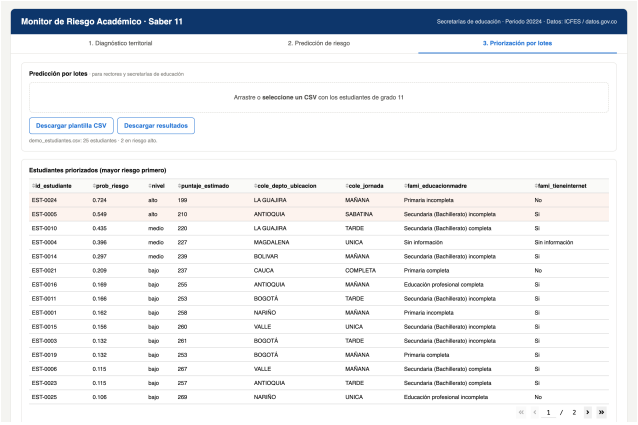


Fig. 7. Pestaña 3 — Priorización por lotes (modo demostración con 25 estudiantes).

VI. Repositorio y evidencia de MLflow en EC2

El código está en https://github.com/mateor0408-sys/proyecto-saber11-dsa: modelos (src/features.py, train.py, run_experiments.sh, select_best.py, report_models.py), tablero (dashboard/), despliegue de MLflow (deploy/) y reportes (docs/). El servidor MLflow corre en la instancia EC2 con IP pública 54.242.29.105 (puerto 5000). Las capturas evidencian la instancia y los experimentos registrados.

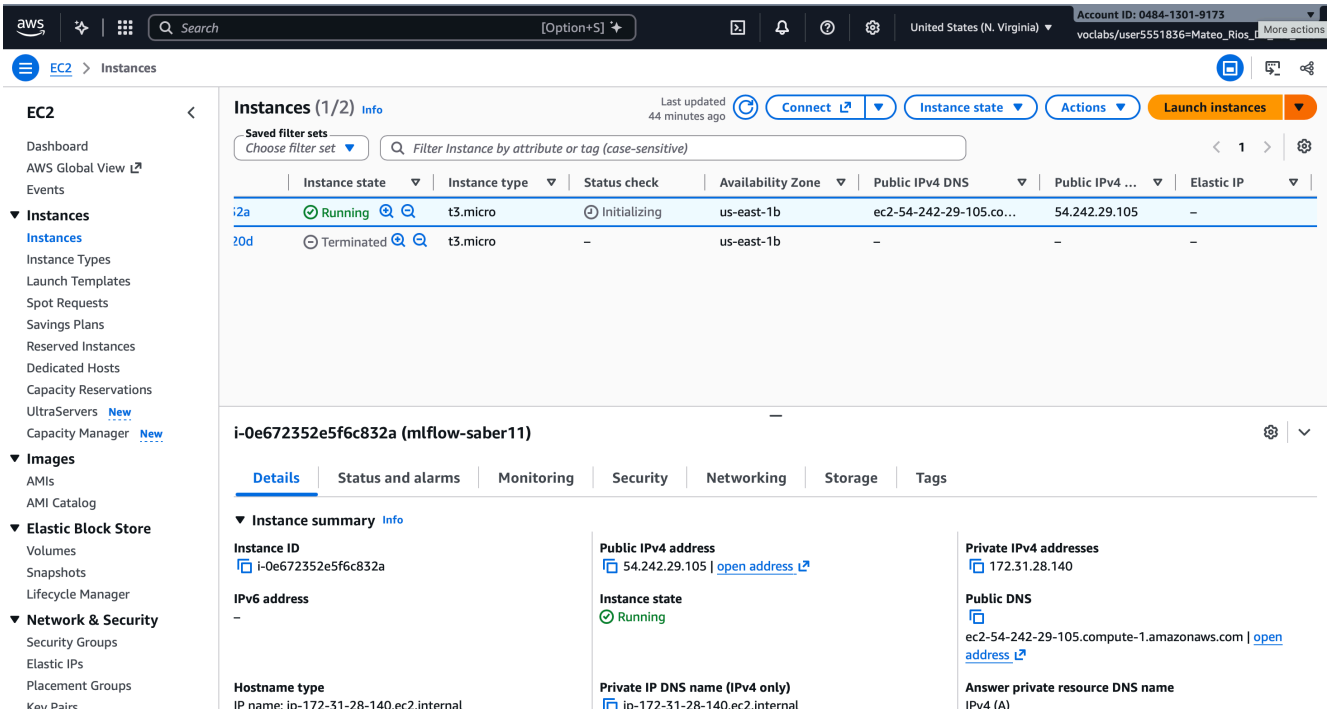


Fig. 8. Instancia EC2 en ejecución (IP 54.242.29.105, usuario visible).

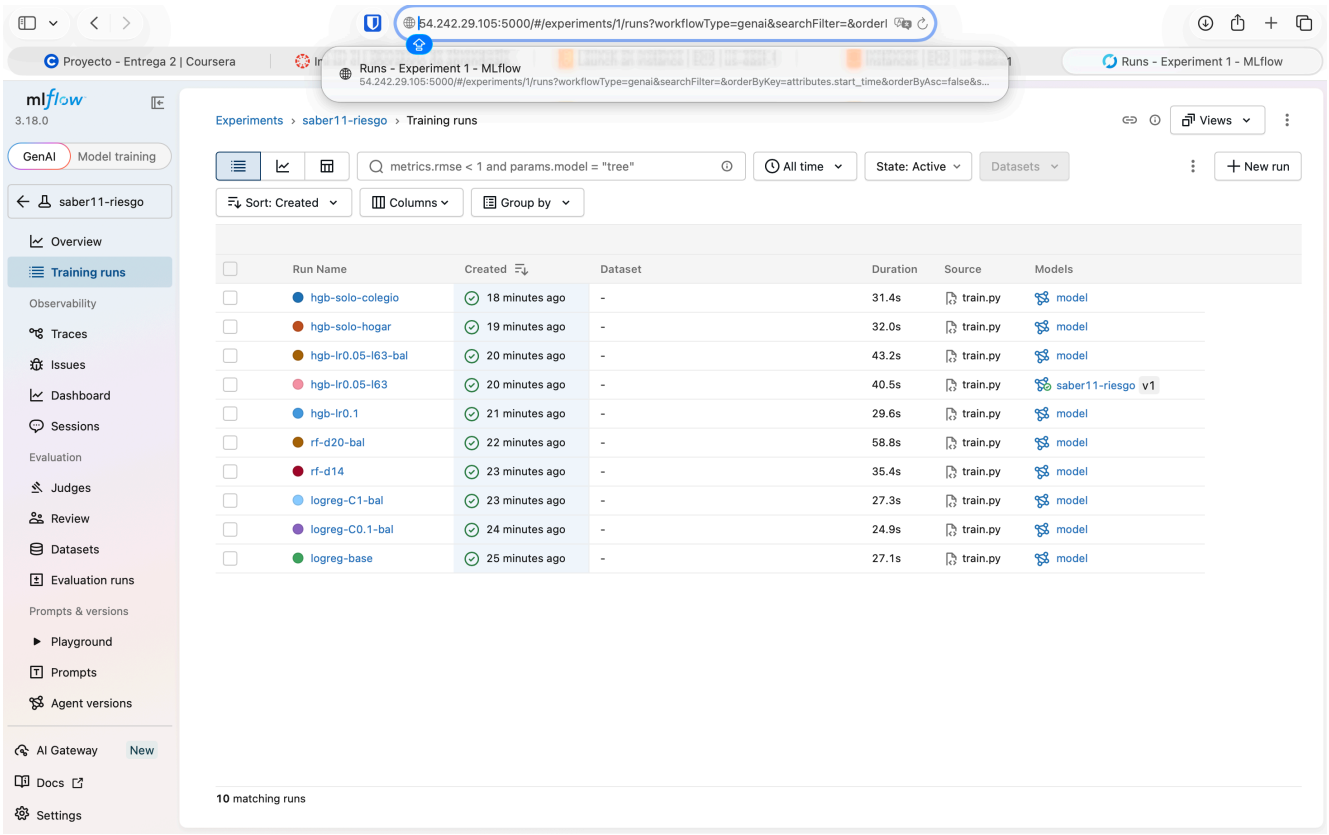


Fig. 9. Runs del experimento saber11-riesgo en MLflow (http://54.242.29.105:5000); el campeón figura como modelo registrado saber11-riesgo v1.

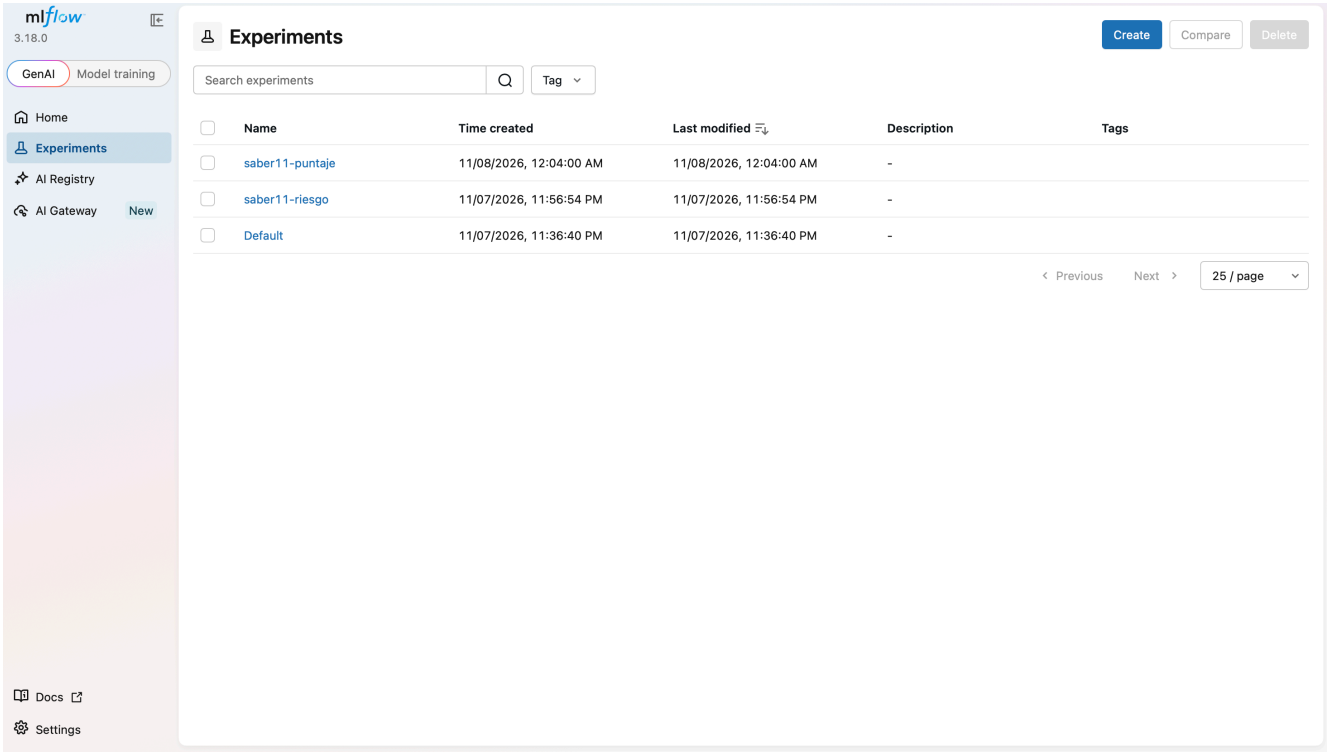


Fig. 10. Experimentos de clasificación (saber11-riesgo) y regresión (saber11-puntaje) en el servidor MLflow.

Referencias

[1] ICFES, “Resultados únicos Saber 11,” Datos Abiertos Colombia, 2024. [En línea]. Disponible: https://www.datos.gov.co/Educaci-n/Resultados-nicos-Saber-11/kgxf-xxbe

[2] M. Zaharia et al., “Accelerating the machine learning lifecycle with MLflow,” IEEE Data Eng. Bull., vol. 41, no. 4, pp. 39–45, 2018.

[3] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[4] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” en Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 3146–3154.

[5] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[6] Plotly Technologies Inc., “Dash.” [En línea]. Disponible: https://dash.plotly.com

[7] T. Saito y M. Rehmsmeier, “The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets,” PLoS ONE, vol. 10, no. 3, e0118432, 2015.